

Chapter 11 - Hash Tables

- 1) A hash table is an effective data structure for implementing dictionaries, i.e. a dynamic set that supports ^{only} the operations INSERT, SEARCH and DELETE.
- 2) A hash table is a generalisation of the simpler notion of an ordinary array. Directly addressing into an ordinary array supports the dictionary operations of INSERT, SEARCH and DELETE in $O(1)$ time.

Directly addressing is applicable when we can afford to allocate an array that has one slot for every possible key.

But when the no. of keys actually stored is small compared to the total no. of possible keys, hash tables become an effective alternative to directly addressing an array. This is because a hash table is a data structure of size proportional to the no. of keys actually stored.

3) DIRECT address tables:-

Direct addressing works well when the universe U of keys is reasonably small. Suppose an application needs a dynamic set in which each element has a key drawn from a universe

$U = \{0, 1, \dots, m-1\}$ (we assume that m is not too large).

A direct-address table is an array $T[0..m-1]$ in which the slot K points to an element in the dynamic set with key K . If no element in the dynamic set has key K then $T[K] = \text{NIL}$.

The dictionary operations then are:-

1) **DIRECT-ADDRESS-SEARCH (T, K) :**

return $T[K]$

2) **DIRECT-ADDRESS-INSERT (T, x)**

$T[\text{Key}[x]] \leftarrow x$

3) **DIRECT-ADDRESS-DELETE (T, x)**

$T[\text{Key}[x]] \leftarrow \text{NIL}$

Each of these operations requires $O(1)$ time.

→ If the elements of the direct-address table are small enough, they themselves can be stored in the direct address table. Moreover, it would be unnecessary to store the keys in the elements, because the indices of the elements can tell about their keys.

4) Hash Tables:-

Direct addressing cannot be used in case the Universe U is large, because storing a table T of size $|U|$ may be impractical given the memory available on a typical computer. Moreover, if the set K of keys actually stored is small relative to U then most of the space allocated for T would be wasted.

Under such circumstances a hash table requires much less space than a direct-address table. The storage requirements are reduced to $\Theta(|K|)$ while searching for an element in the hash table still requires $O(1)$ time. But this bound is for the average time whereas for direct addressing it holds for the worst case time.

→ In case of hashing an element with key k is stored in slot $h(k)$. It is called a hash-function that maps the universe U of keys into the slots of a hash table $T[0 \dots m-1]$, i.e.:-

$$h: U \rightarrow \{0, 1, \dots, m-1\}$$

→ We say that an element with key k hashes to slot $h(k)$. We also say that $h(k)$ is the hash value of key k .

→ When two keys hash to the same slot we say a collision has occurred. We can choose a suitable hash function to minimize the number of collisions. But avoiding collisions altogether is impossible as $|U| > m$ (at least 2 keys of the universe hash to the same slot).

5) Collision resolution by chaining:-

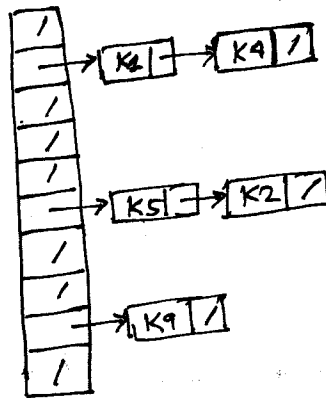
→ In chaining we put all the elements that hash to the same slot in a linked list. Slot j of the array T contains a pointer to the head of the linked list of all elements that hash to j ; if there are no such elements slot j

contains NIL.

The dictionary operations then are :-

1) CHAINED-HASH-INSERT(T, x)

insert x at the head of the list $T[h(\text{Key}[x])]$



2) CHAINED-HASH-SEARCH(T, K)

search for an element with key K in the list $T[h(K)]$

3) CHAINED-HASH-DELETE(T, x)

delete x from the list $T[h(\text{Key}[x])]$

→ The worst case running time for insertion is $O(1)$ assuming that the element x being inserted is not already present in the table.

→ Deletion of an element x can be accomplished in $O(1)$ time if the lists are doubly linked.

→ **Load factor(α):** Load factor of a hash table is defined as the average no. of elements stored in a chain. For a hash table T with m slots and n elements

$$\alpha = n/m$$

→ **Simple uniform hashing:** In simple uniform hashing every element is equally likely to hash into any of the m slots independently of ^{where} any other element has to.

→ In the worst case all the n keys hash to the same slot creating a list of length n . The worst case time for searching is thus $\Theta(n)$ plus the time to compute the hash function.

→ The average performance of hashing depends upon how well the hash function h distributes the set of keys among the m slots on an average.

If the length of the list $T[j]$ is denoted by $|T[j]|$ ~~where~~ n_j , then

→ no. of elements in the hash table

$$= n = n_0 + n_1 + n_2 + \dots + n_{m-1}$$

→ average value of $n_j = E[n_j] = \alpha = n/m$

→ We assume that the hash value $h(K)$ can be computed in $O(1)$ time.

→ The time required to search for an element with key K depends linearly on the length $n_{h(K)}$ of the list $T[h(K)]$.

We consider 2 cases:-

1) In a hash table in which collisions are resolved by chaining an unsuccessful search takes expected time $\Theta(1+\alpha)$, under the assumption of simple uniform hashing:-

expected time to search unsuccessfully for key K = expected time to search to the end of list $T[h(K)]$

$$\text{But, } E[\Theta_{h(K)}] = \alpha$$

Thus, total time required (including the time for computing $h(K)$) is $\Theta(1+\alpha)$

2) In a hash table in which collisions are resolved by chaining, a successful search takes $\Theta(1+\alpha)$, on the average, under the assumption of simple uniform hashing:-

→ We assume that the element being searched for is equally likely to be any of the n elements stored in the table.

→ The no. of elements examined during a successful search for an element x is 1 more than the no. of elements that appear before x in x 's list.

→ Elements before x in the list were all inserted after x was inserted, because new

elements are placed at the front of the list.

→ Let x_i = the i th element inserted into the table. ($i = 1, 2, \dots, n$). Let $K_i = \text{Key}[x_i]$.

→ For keys K_i and K_j we define an indicator random variable $X_{ij} = I\{h(K_i) = h(K_j)\}$. Under the assumption of simple uniform hashing, we have $P\{h(K_i) = h(K_j)\} = 1/m$.

$$\therefore E[X_{ij}] = 1/m.$$

→ The expected no. of elements examined in a successful search is =

$$E \left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij} \right) \right]$$

$$= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E[X_{ij}] \right) \quad \left\{ \begin{array}{l} \text{By linearity} \\ \text{of} \\ \text{expectation} \end{array} \right.$$

$$= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n 1/m \right)$$

$$= \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m} \right) = \frac{1}{n} \sum_{i=1}^n 1 + \frac{1}{nm} \sum_{i=1}^n (n-i)$$

$$= 1 + \frac{1}{nm} \left(\sum_{i=1}^n n - \sum_{i=1}^n i \right)$$

$$= 1 + \frac{1}{2m} \left(n^2 - \frac{n(n+1)}{2} \right)$$

$$= 1 + \frac{n-1}{2m} = 1 + \frac{n}{2m} - \frac{1}{2m}$$

$$= 1 + \frac{\alpha}{2} - \frac{\alpha}{2m}$$

$\therefore$ Total time required for a successful search (including the time for computing the hash function is) $= \Theta\left(2 + \frac{\alpha}{2} - \frac{\alpha}{2m}\right)$

$$= \Theta(1 + \alpha)$$

$\rightarrow$ If the no. of hash table ^{slots (m)} is at least proportional to the no. of elements in the table (~~m~~), we have $n = O(m)$ and $\alpha = \frac{n}{m}$

$$= \frac{O(m)}{m} = O(1).$$

Thus, searching takes constant time on average.

$\rightarrow$ All 3 dictionary operations can be supported in $O(1)$ time on average.

6) Hash Functions:- A good hash function should satisfy the assumption of simple uniform hashing.

Most hash functions assume that the universe of keys is the set $N = \{0, 1, 2, \dots\}$ of natural numbers.

Thus, if the keys are not natural numbers, a way is found to interpret them as natural numbers. For example, a character string can be interpreted as an integer expressed in suitable radix notation.

eg:- $pt = (112, 116)$ in ASCII notation =
expressed as a radix-128 integer pt
becomes $= 112 \cdot 128 + 116 = 14452$.

1) The division method:-

In the division method the hash function is $h(K) = K \bmod m$, where m is the total no. of available slots.

When using the division method we usually avoid certain values of m :-

- ① m should not be a power of 2, for if $m = 2^p$ then $h(K) = K \bmod 2^p$ is just the p lowest order bits of K . It is better to make the hash function dependent on

all the bits of the key.

- (ii) choosing $m = 2^p - 1$ when K is a character string interpreted in radix 2^p may be a poor choice, because permuting the characters of K does not change its hash value.

~~(iii)~~ A prime number not too close to an exact power of 2 is often a good choice for m .

2) The multiplication method:-

The hash function is defined as

$$h(K) = \lfloor m(KA \bmod 1) \rfloor$$

where, A is a constant in the range $(0,1)$
i.e. $0 < A < 1$

$KA \bmod 1$ means the fractional part of KA
i.e. $KA - \lfloor KA \rfloor$

→ the advantage of multiplication method is that the value of m is not critical. We typically choose m to be a power of 2 i.e. $m = 2^p$ so that we can easily implement the function on most computers.

→ Knuth suggests that the method works best with a choice of A close to $\frac{\sqrt{5}-1}{2} \approx 0.618$

7) Open Addressing:-

→ In open addressing all elements are stored in the hash table itself. That is every table entry either contains an element of the dynamic set or NIL. There are no lists and no elements stored outside the table as there are in chaining. Thus, in open addressing the hash table can "fill up" so that no further insertions can be made, the load factor α can never exceed 1.

→ When searching for an element we systematically scan the table slots until the desired element is found or it is clear that the element is not in the table. The advantage of open addressing is that it avoids pointers altogether. The memory freed by not storing pointers provides the hash table with larger number of slots for the same amount of memory, potentially yielding fewer collisions and faster retrieval.

→ To perform insertion using open addressing we successively scan or probe the hash table until we find an empty slot in which to put the key.

The slots are not probed in the sequence $0, 1, \dots, m-1$ as it would require $\Theta(m)$ search time.

The sequence of slots probed depends upon the key being inserted. We extend the hash function to include the probe number (which starts at 0). Thus, the hash function becomes: -

$$h: U \times \underbrace{\{0, 1, 2, \dots, m-1\}}_{\text{probe number}} \rightarrow \underbrace{\{0, 1, 2, \dots, m-1\}}_{\text{slots}}$$

Thus, the probe sequence is $\langle h(K, 0), h(K, 1), h(K, 2), \dots, h(K, m-1) \rangle$ which is some permutation of $\langle 0, 1, 2, \dots, m-1 \rangle$

→ In the following procedures we assume that the elements in the hash table are just keys with no satellite data.

HASH-INSERT (T, K)

```
1  i ← 0
2  repeat  j ← h(K, i)
3          if T[j] = NIL
4              then T[j] ← K
5                  return
6          else i ← i + 1
7  until i = m
8  error "hash table overflow"
```

HASH-SEARCH (T, K)

```
1   $i \leftarrow 0$   
2  repeat  $j \leftarrow h(K, i)$   
3      if  $T[j] = K$   
4          then return  $j$   
5       $i \leftarrow i + 1$   
6  until  $T[j] = \text{NIL}$  or  $i = m$   
7  return NIL
```

→ Deletion from an open address hash table is difficult. When we delete a key from slot i , we cannot simply mark that slot empty by storing NIL in it. Doing so would make it impossible to retrieve any key K during whose insertion we probed slot i and found it occupied.

One solution is to mark the slot by storing in it a special value of DELETED instead of NIL. We then need to modify the HASH-INSERT procedure to treat a DELETED slot as empty so that a new key can be inserted there.

No modification of HASH-SEARCH is needed as it will pass over the DELETED slots. But then the search times would be

longer depend upon the load factor α .

For this reason chaining is more commonly used as a collision resolution technique when keys must be deleted.

→ In open addressing we desire "uniform hashing" i.e. each key is equally likely to have any of the m permutations of $\langle 0, 1, \dots, m-1 \rangle$ as its probe sequences.

Techniques used to compute the probe sequences required for open addressing:-

1) Linear Probing:

Suppose an auxiliary hash function $h': U \rightarrow \{0, 1, \dots, m-1\}$. Linear probing uses the hash function

$$h(K, i) = (h'(K) + i) \bmod m \quad \text{for } i \in [0, m-1]$$

→ The first slot probed is $T[h'(K)]$, then $T[h'(K)+1]$ and so on till $T[m-1]$. Then we wrap around to slots $T[0]$, $T[1]$ and so on until we finally probe the slot $T[h'(K)-1]$.

→ Because the initial probe determines the entire probe sequence, there are only m distinct probe sequences. The assumption of uniform hashing is not fulfilled since m sequences are not generated.

→ Linear probing suffers from the problem of primary clustering. Long runs of occupied slots build up increasing the average search time. Clustering arises since an empty slot preceded by i full slots gets filled next with probability $(i+1)/m$. Long runs of occupied slots tend to get longer and the average search time increases.

2) Quadratic probing:-

quadratic probing uses a hash function of the form:-

$$h(k) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

where, c_1 and c_2 are constants $\neq 0$.

Quadratic probing works better than linear probing.

→ Quadratic probing suffers from secondary clustering. If two keys have the same initial probe position i.e. if for $k_1 \neq k_2$ $h(k_1, 0) = h(k_2, 0)$ then for all $i \in [1, m-1]$ $h(k_1, i) = h(k_2, i)$. This leads to secondary clustering.

→ Since the initial probe determines the entire probe sequence, only m sequences are possible and true uniform hashing is not achieved.

3) Double Hashing:-

Double Hashing uses a hash function of the form :-

$$h(K, i) = (h_1(K) + i h_2(K)) \bmod m$$

where, h_1 and h_2 are auxiliary hash functions.

→ For the entire hash table to be searched the value $h_2(K)$ must be relatively prime* to the hash table size m .

This may be achieved in 2 ways:-

- (i) let m be a power of 2 and design h_2 so that it always produces an odd no.
- (ii) let m be prime and design h_2 so that it always returns a positive integer less than m .

Double hashing produces $O(m^2)$ probe sequences since each possible $(h_1(K), h_2(K))$ pair yields a distinct probe sequence.

Hence double hashing is closer to ideal uniform hashing than linear probing or quadratic probing.

*→ 2 integers are said to be relatively prime or co-prime if they do not have any factor in common, other than 1.